

# Virtual Memory - Principal Engineer

Visual Revision Guide | 1-2 line answers + tiny examples + memory hooks

Use this in the interview: Give the “Say” answer first. Draw the tiny diagram only if the interviewer probes. The green line is the one sentence to memorize.

## 1. Translation, TLB and Faults

### 1. Explain virtual memory from CPU instruction to DRAM access.

**Say:** A load/store produces a virtual address. The MMU checks the TLB; on a hit it gets the physical address, then checks CPU caches, and only a cache miss may reach DRAM.

CPU instruction -> VA -> TLB -> PA -> L1/L2/L3 -> DRAM

**Remember: Translation first; DRAM is only needed if the data is not already cached.**

### 2. Why do we need virtual memory?

**Say:** It gives every process isolation and protection while letting the OS map virtual pages flexibly to RAM, files, shared pages or swap.

Process A VA --\  
> different physical pages  
Process B VA --/

**Remember: Virtual memory = isolation + protection + flexible mapping, not just “more memory than RAM”.**

### 3. What exactly is stored in a page table entry?

**Say:** A PTE mainly stores the physical page-frame number plus flags such as present, writable, user/kernel, executable, accessed and dirty.

PTE = [ PFN | Present | R/W | User | NX | Accessed | Dirty ]

**Remember: PTE = where the page is + what you are allowed to do with it.**

### 4. What happens during a TLB miss?

**Say:** The translation is not cached, so the CPU walks the page tables. If the PTE is valid it fills the TLB and continues; otherwise a page fault may occur.

TLB MISS -> Page-table walk -> valid PTE? -> fill TLB

**Remember: TLB miss normally stays in hardware; it does not automatically enter the kernel.**

### 5. TLB miss vs page fault?

**Say:** TLB miss means the translation is not cached. Page fault means the current page-table state or permissions cannot satisfy the access, so the CPU traps to the kernel.

TLB miss -> page-table walk  
Page fault -> kernel handler

**Remember: TLB miss = translation lookup. Page fault = OS help required.**

### 6. Explain a page fault end-to-end.

**Say:** CPU traps to the kernel; the kernel validates the address, obtains/allocates the page, updates the PTE, invalidates stale TLB state if needed, and retries the instruction. Invalid access becomes SIGSEGV.

Access -> fault -> kernel -> validate -> map page -> retry  
\-> invalid -> SIGSEGV

**Remember: A page fault is a mechanism; it is not always an error.**

### 7. Minor vs major page fault?

**Say:** A minor fault is resolved without reading the needed page from storage. A major fault requires storage I/O, so it is far more expensive.

Minor: fault -> RAM work -> resume  
Major: fault -> DISK/SSD -> RAM -> resume

**Remember: Major = storage on the critical path.**

## 8. What causes segmentation faults?

**Say:** An access to an unmapped address or a protection violation can become SIGSEGV—for example null/dangling pointers, stack overflow, or writing read-only memory.

```
bad pointer -> CPU protection fault -> kernel -> SIGSEGV
```

**Remember:** Out-of-bounds does not always segfault; it may silently corrupt another valid page.

## 2. Allocation, Demand Paging and COW

### 9. Why can malloc(1 GB) succeed without consuming 1 GB RAM?

**Say:** malloc can reserve virtual address space without immediately backing every page with RAM. Physical pages are generally supplied as the program touches them.

```
malloc(1 GB) -> 1 GB virtual range  
Touch pages -> physical RAM grows
```

**Remember:** malloc reserves address space; touching pages creates real memory pressure.

### 10. What happens on the first access to newly allocated memory?

**Say:** An untouched anonymous page commonly faults. The kernel supplies a zeroed physical page, maps it, and retries the instruction.

```
malloc -> untouched page  
write -> minor fault -> zero page allocated -> continue
```

**Remember:** First touch converts virtual reservation into physical backing.

### 11. Explain demand paging.

**Say:** Pages are allocated or loaded only when first accessed rather than eagerly at mapping time.

```
map 1 GB: [.....]  
touch 8 KB: [##.....]
```

**Remember:** Pay for pages on demand, not up front.

### 12. Explain fork() using Copy-on-Write.

**Say:** After fork, parent and child initially share physical pages. A write causes a COW fault and the writer gets a private copy.

```
Parent --\  
> Page A (shared/read-only)  
Child --/  
Child writes -> Child Page B
```

**Remember:** fork shares first, copies only on write.

### 13. Why is fork() relatively cheap initially?

**Say:** It avoids copying the whole user address space; the kernel mostly duplicates metadata/page-table state and keeps data pages shared via COW.

```
1 GB process -> fork != copy 1 GB immediately
```

**Remember:** Cheap initially, but page-table work and later COW writes can still be expensive.

### 14. What if parent and child modify the same COW page simultaneously?

**Say:** Both writes fault on their own mappings. Kernel synchronization makes the COW transition safe, and each process ends with its own writable physical page.

```
Shared Page A  
Parent write -> Page P  
Child write -> Page C
```

**Remember:** COW preserves process isolation even with concurrent writers.

## 3. Sharing, mmap and Page Cache

## 15. How does shared memory work across processes?

**Say:** The OS maps the same physical frames into two processes. They can use different virtual addresses but see the same underlying bytes.

```
Proc A VA 0x1000 --\
> Physical Page X
Proc B VA 0x9000 --/
```

**Remember:** Shared memory shares data, not synchronization—use mutex/futex/atomics.

## 16. Can two processes have the same virtual address?

**Say:** Yes. The same virtual address can map to different physical pages because each process has its own page tables.

```
A: VA 0x1000 -> PA 10
B: VA 0x1000 -> PA 99
```

**Remember:** Virtual addresses are meaningful only inside an address space.

## 17. Can two different virtual addresses point to the same physical address?

**Say:** Yes. Shared memory, shared libraries, file mappings and pre-write COW can create such aliases.

```
VA 0x1000 --\
> same PA 42
VA 0x9000 --/
```

**Remember:** Different VA can intentionally alias the same physical page.

## 18. How does mmap() work?

**Say:** mmap creates a virtual range backed by a file or anonymous memory. Pages are usually populated lazily through page faults.

```
mmap(file) -> virtual range -> fault -> page cache -> PTE
```

**Remember:** mmap creates mappings; faults populate them.

## 19. mmap() vs malloc()?

**Say:** malloc is a user-space allocator API for objects; mmap is an OS mapping primitive. Allocators themselves may use mmap for large allocations.

```
malloc -> allocator -> brk/mmap -> VM
mmap -----> VM
```

**Remember:** malloc is allocation policy; mmap is a VM mechanism.

## 20. What is the page cache?

**Say:** It is RAM used by the kernel to cache file contents, reducing repeated disk I/O. Buffered I/O and file-backed mmap commonly use it.

```
SSD <-> [ PAGE CACHE in RAM ] <-> process
```

**Remember:** Low “free” RAM can be healthy because spare RAM is useful as file cache.

## 21. How do memory-mapped files interact with page cache?

**Say:** A file-backed mmap normally maps page-cache pages. MAP\_SHARED writes dirty the shared cached page; MAP\_PRIVATE writes use COW.

```
file -> page cache -> mmap PTE
MAP_SHARED: write shared
MAP_PRIVATE: COW
```

**Remember:** mmap does not require a separate duplicate copy of the file data.

## 22. Why are page tables multi-level?

**Say:** A giant flat table would waste huge amounts of memory for sparse address spaces. Multi-level tables allocate lower levels only for mapped regions.

```
Root -> L2 -> L3 -> L4 -> 4 KB page
unused branches need not exist
```

**Remember:** Multi-level paging trades extra walk steps for much lower page-table memory.

# 4. Memory Pressure and Reclaim

## 23. What is RSS vs VSZ?

**Say:** VSZ is mapped virtual address space; RSS is the resident portion currently backed by physical RAM. A huge VSZ does not imply huge RAM use.

```
VSZ: [=====]
RSS: [####.....]
```

**Remember: VSZ = address space; RSS = resident memory.**

## 24. What happens under memory pressure?

**Say:** Linux reclaims cold pages: clean file cache can be dropped, dirty pages may be written back, anonymous pages may be swapped, and severe pressure can end in OOM.

```
Pressure -> reclaim -> writeback/swap -> stalls -> OOM
```

**Remember: Memory pressure hurts latency before it necessarily causes OOM.**

## 25. What is swapping?

**Say:** Anonymous pages are moved from RAM to swap so RAM can be reused; later access causes them to be read back.

```
RAM page -> SWAP
later access -> major fault -> RAM
```

**Remember: Swap buys capacity but can add large latency.**

## 26. What is thrashing?

**Say:** The working set does not fit in memory, so the system repeatedly evicts and faults pages instead of doing useful work.

```
use A -> evict B -> need B -> evict A -> repeat
```

**Remember: Thrashing = paging/reclaim dominates useful execution.**

## 27. How does the OS choose pages to reclaim?

**Say:** Linux approximates which pages are cold using access history and reclaim lists/generations; clean cold pages are easier to reclaim than hot or dirty pages.

```
HOT pages -> keep
COLD pages -> reclaim
```

**Remember: Think “approximate recency + reclaim cost,” not perfect LRU.**

## 28. What are dirty and accessed bits?

**Say:** Accessed says a page has been referenced; dirty says it has been written. The OS uses these signals for reclaim and writeback decisions.

```
read -> Accessed=1
write -> Accessed=1, Dirty=1
```

**Remember: Accessed helps estimate hotness; dirty means data cannot simply be discarded.**

## 29. Internal vs external fragmentation?

**Say:** Internal fragmentation is wasted space inside allocated units; external fragmentation is free space split into pieces that cannot satisfy a large contiguous request.

```
Internal: [used|waste]
External: [free][used][free][used]
```

**Remember: Internal = waste inside; external = holes between allocations.**

# 5. TLB Coherence, Huge Pages and NUMA

## 30. What happens when a page-table entry changes?

**Say:** After the kernel changes a mapping or permissions, any stale TLB translation must be invalidated before CPUs can safely use the new PTE.

```
PTE changes -> invalidate stale TLB -> use new mapping
```

**Remember: Updating memory is not enough; cached translations must stay coherent.**

### 31. What is a TLB shutdown?

**Say:** When another CPU may cache a changed mapping, the kernel asks that CPU to invalidate the stale TLB entry, typically using inter-processor coordination.

```
CPU0 changes PTE -> IPI -> CPU1/2/3 invalidate TLB
```

**Remember: TLB shutdown = cross-core translation invalidation.**

### 32. Why can TLB shutdowns hurt multicore scalability?

**Say:** They interrupt other cores, add synchronization, and throw away useful translation-cache entries. Frequent mapping changes can therefore scale poorly.

```
1 mapping change -> notify N CPUs -> N invalidations
```

**Remember: More cores can make global VM changes more expensive.**

### 33. Why do huge pages improve performance?

**Say:** A 2 MB page covers 512 times more memory than a 4 KB page per translation, increasing TLB reach and reducing page-table walks.

```
4 KB: [ ][ ][ ][ ] ... many translations
2 MB: [=====] one large translation
```

**Remember: Huge page = more memory covered per TLB entry.**

### 34. What are disadvantages of huge pages?

**Say:** They can waste memory through internal fragmentation, need larger contiguous physical regions, and can make allocation, compaction, migration or COW more expensive.

```
Small use inside 2 MB page -> wasted space
```

**Remember: Huge pages trade granularity/flexibility for translation efficiency.**

### 35. What is NUMA and why does placement matter?

**Say:** On NUMA machines, memory attached to the local CPU socket is faster than remote-node memory. Bad thread/page placement increases latency and interconnect traffic.

```
CPU0 -> Local RAM : fast
CPU0 -> CPU1 RAM : slower
```

**Remember: Keep threads close to their working-set memory.**

### 36. Why can a program be slow despite apparently available memory?

**Say:** The problem may be page faults, reclaim stalls, NUMA remote access, TLB misses, cgroup limits, compaction or page-cache misses—not total free RAM.

```
Available RAM != memory is local/resident/cheap to access
```

**Remember: Measure the memory path causing latency, not only free/available memory.**

## 6. Context Switches and Protection

### 37. What happens to page tables during a context switch?

**Say:** For a different process address space, the kernel switches the page-table root. It does not copy all page tables during every switch.

```
Process A -> root A
context switch
Process B -> root B
```

**Remember: Switch the root, not the whole page table.**

### 38. Does every context switch flush the TLB?

**Say:** No. Same-address-space thread switches do not need it, and PCID/ASID-capable CPUs can keep translations from multiple address spaces tagged in the TLB.

```
TLB: [A:VA->PA] [B:VA->PA]
tagged by address space
```

**Remember: Modern CPUs avoid full TLB flushes when possible.**

### 39. What are ASIDs/PCIDs?

**Say:** They tag TLB entries by address space, allowing translations from different processes to coexist safely and survive context switches.

```
TLB entry = [ASID/PCID | VA | PA]
```

**Remember: Tags reduce destructive TLB flushes.**

### 40. How are memory permissions enforced?

**Say:** The MMU checks PTE permissions—such as writable, executable and user/kernel—on memory accesses. Violations trap to the kernel.

```
PTE: R=1 W=0  
CPU write -> protection fault
```

**Remember: OS sets policy in PTEs; hardware enforces it.**

## 7. Principal-Level Production Scenarios

### 41. How would you diagnose excessive page faults?

**Say:** First separate minor from major faults and correlate them with latency. Then identify whether the source is demand-zero/COW, mmap file I/O, swap, reclaim or working-set growth.

```
pidstat/perf -> minor vs major -> mapping/cause -> I/O/reclaim
```

**Remember: Do not optimize “page faults” until you know which kind and whether they are on the critical path.**

### 42. How would you diagnose increasing RSS?

**Say:** Break RSS down by anonymous/file-backed and private/shared mappings using smaps; then compare application live bytes with allocator retained memory and caches.

```
RSS growth -> smaps -> heap? mmap? stacks? shared? allocator?
```

**Remember: RSS growth is a symptom, not proof of a leak.**

### 43. Leak vs page-cache growth vs allocator fragmentation?

**Say:** Leak: application live/private anonymous bytes keep growing. Page cache: file-backed/reclaimable kernel memory grows. Fragmentation: allocator RSS stays high while live allocations are much lower.

```
Live bytes ~= RSS? leak candidate  
Live bytes << allocator RSS? fragmentation  
File cache grows? page cache
```

**Remember: Compare application, allocator and kernel layers separately.**

### 44. Low CPU but very high latency under memory pressure—what investigate?

**Say:** Look for waiting rather than compute: major faults, swap, direct reclaim, PSI memory stalls, storage latency, dirty writeback, cgroup pressure and NUMA issues.

```
low CPU + high latency -> WAITING  
-> reclaim / I/O / faults
```

**Remember: Low CPU does not mean healthy; requests may be blocked on memory or I/O.**

### 45. 100-GB DB slower after huge pages -> 4-KB pages. Why?

**Say:** TLB reach drops dramatically, causing many more TLB misses/page-table walks and larger page tables. Translation stalls can become material for a large random-access working set.

```
2 MB page = 512 x 4 KB pages  
100 GB: ~51K 2MB mappings vs ~26M 4KB mappings
```

**Remember: For large working sets, huge pages can turn translation from a bottleneck into a minor cost.**

## One-Page Memory Map

| Concept       | One line to remember                                                 |
|---------------|----------------------------------------------------------------------|
| TLB miss      | Translation is not cached; hardware walks page tables.               |
| Page fault    | Current mapping/permission cannot satisfy access; kernel handles it. |
| Minor / Major | Minor = no required storage read; Major = storage I/O.               |
| malloc        | Reserve virtual memory first; physical pages arrive on touch.        |
| fork/COW      | Share first; copy on write.                                          |
| Shared memory | Different virtual addresses can map the same physical page.          |
| mmap          | Create a mapping; faults populate pages.                             |
| Page cache    | RAM cache of file contents.                                          |
| RSS / VSZ     | Resident RAM / mapped virtual address space.                         |
| Thrashing     | Paging/reclaim dominates useful work.                                |
| TLB shutdown  | Invalidate stale translations on other CPUs.                         |
| Huge pages    | More TLB reach, less translation overhead.                           |
| NUMA          | Local memory is cheaper than remote memory.                          |
| PCID/ASID     | Tag TLB entries so context switches need fewer flushes.              |
| RSS diagnosis | Classify memory before calling it a leak.                            |

**Best interview technique: answer in 15-20 seconds, stop, and let the interviewer choose the depth. If they probe, draw the diagram and connect the concept to performance or production behavior.**